

M06 - Git + GitHub + Linux/WSL Engineering Foundations

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Terminal rule

Every destructive-looking command is confined to a disposable sandbox or Stage 06 practice repository. Read `pwd/status/diff` before acting.

Contents

- DE06-01** - Verify WSL2 and choose a safe Linux workspace
- DE06-02** - Linux navigation and safe file operations
- DE06-03** - Pipes, redirection, command history and inspection
- DE06-04** - Permissions, packages, sudo and command safety
- DE06-05** - Git state: init, status, add, commit, diff and log
- DE06-06** - Branches, merges and real conflict resolution
- DE06-07** - GitHub remotes: push, fetch, pull and tracking
- DE06-08** - Pull requests and review workflow
- DE06-09** - Python virtual environments inside a repository
- DE06-10** - `.gitignore`, secrets and repository hygiene
- DE06-11** - restore, revert, reflog and safe history repair
- DE06-12** - Shell scripts, exit codes and environment-driven commands
- DE06-13** - Repository handoff and clean-clone reproducibility

DE06-01 - Verify WSL2 and choose a safe Linux workspace

What you will be able to do

Know whether WSL actually works, understand Windows-vs-Linux paths, and create one safe course workspace.

Why this matters

Later tools often use a Linux-like shell. Reinstalling WSL whenever a command looks unfamiliar wastes time and can create extra environments. First verify reality.

Picture it this way

WSL is like a Linux workshop built into your Windows building. You can reach Windows files through /mnt/c, but the workshop has its own normal home folder and tools.

Start from zero

- WSL means **Windows Subsystem for Linux**. A distro such as Ubuntu runs a Linux user space integrated with Windows.
- From Windows PowerShell, **wsl --status** and **wsl -l -v** show whether a distro exists and whether it uses WSL 2.
- Inside Ubuntu/WSL, **whoami**, **pwd** and **uname -a** prove which user/environment you are in.
- Your Linux home is commonly /home/USERNAME and written as ~. Windows C: is mounted under /mnt/c.
- For course repositories, use a Linux-home workspace such as **~/de-course** by default. Use /mnt/c deliberately for exchange, not because paths look familiar.
- If WSL is not installed, use the official Microsoft install route only after verification. Do not run reinstall commands just because a tutorial says so.
- Bash keeps command history. **history** lists it; Ctrl+R searches backwards. Commands may also persist in ~/.bash_history, so never type secrets directly into commands unless you understand the exposure.

Teacher demonstration

```
# In Windows PowerShell:
wsl --status
wsl -l -v
# Then open your Ubuntu/WSL terminal:
whoami
pwd
uname -a
mkdir -p ~/de-course/m06
cd ~/de-course/m06
pwd
```

Read the commands like English

- The first two commands run in Windows PowerShell, not inside Linux.
- Inside WSL, ~ means your Linux home folder.
- mkdir -p creates parent folders if needed and does not fail merely because the folder already exists.
- The final pwd should end in /de-course/m06.

Expected check

check	what you need
WSL state	a distro available and preferably VERSION 2

check	what you need
Linux identity	whoami returns your Linux username
workspace	~/de-course/m06
history	history/Ctrl+R can retrieve earlier commands

Common beginner traps

- Running wsl --install before checking whether WSL already exists.
- Using /mnt/c for every repository without understanding cross-filesystem differences.
- Typing a password/token in a command and forgetting shell history can retain it.

Now you do a similar task

Verify WSL without changing system settings. Create ~/de-course/m06 and save a text file named environment_evidence.txt containing the outputs of whoami, pwd and uname -a (you may copy them manually).

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What is the difference between C:\Users\... and ~/de-course/... when you are working inside WSL?

Answer in your own words before continuing.

DE06-02 - Linux navigation and safe file operations

What you will be able to do

Predict exactly which file/folder a shell command will touch before you create, copy, move or delete it.

Why this matters

Terminal commands are powerful because they do not ask many questions. Path awareness is the safety skill.

Picture it this way

A relative path is 'the room next door' - its meaning depends on where you currently stand. An absolute path is the full street address.

Start from zero

- **pwd** shows the current working directory. Use it as a safety check.
- **ls -la** shows files including hidden names like `.gitignore`.
- A relative path such as `sandbox/note.txt` starts from the current directory. An absolute path starts from `/`.
- **mkdir** creates directories; **touch** creates an empty file or updates its timestamp.
- **cp** copies; **mv** moves/renames; **rm** removes. Removal normally bypasses a desktop Recycle Bin.
- For this module, destructive practice stays inside a disposable sandbox. Do not practise recursive deletion on real course folders.

Teacher demonstration

```
cd ~/de-course/m06
pwd
mkdir -p sandbox/notes
touch sandbox/notes/original.txt
cp sandbox/notes/original.txt sandbox/notes/copy.txt
mv sandbox/notes/copy.txt sandbox/notes/moved.txt
ls -la sandbox/notes
rm sandbox/notes/moved.txt
ls -la sandbox/notes
```

Read the commands like English

- **pwd** confirms the base folder first.
- All modified paths begin with `sandbox/`, so the blast radius is intentionally small.
- **cp** leaves `original.txt` and creates `copy.txt`.
- **mv** renames/moves `copy.txt` to `moved.txt`.
- **rm** deletes only `moved.txt`. `original.txt` remains.

Expected check

checkpoint	expected
before remove	original.txt + moved.txt
after remove	original.txt only
outside sandbox	unchanged

Common beginner traps

- Running `rm` from a directory you did not verify.
- Copying an internet command containing `rm -rf` without understanding its target.
- Assuming relative paths mean the same thing after `cd` changes your location.

Now you do a similar task

Create `sandbox/project/a.txt`, copy it to `b.txt`, move `b.txt` into `sandbox/archive/`, remove only the moved copy, then use `find sandbox -maxdepth 3 -type f` to prove `a.txt` remains.

How to prove it

- Save commands + observed output.
- Save `pwd`/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended `sandbox/practice` repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Before any destructive command, what two path checks can you perform?

Answer in your own words before continuing.

DE06-03 - Pipes, redirection, command history and inspection

What you will be able to do

Combine small shell tools, redirect output deliberately, and recover earlier commands without guessing.

Why this matters

Fast terminal inspection helps catch wrong paths, row-count surprises and repeated values before heavier code runs.

Picture it this way

A pipe is a conveyor belt: the text output from one tool becomes input to the next. Redirection sends output into a file instead of the screen.

Start from zero

- The pipe operator `|` sends stdout from one process into another.
- `>` replaces a file's contents; `>>` appends. The difference matters.
- **head**, **tail**, **grep**, **wc**, **sort** and **uniq** are useful quick-inspection tools.
- Shell line counts are not automatically business-row counts: a CSV header adds a line, and complex quoted data can contain newlines.
- **history** displays command history; `Ctrl+R` searches it interactively.
- History is useful for learning/repeating commands, but it can also retain sensitive command text.

Teacher demonstration

```
cd ~/de-course/m06
printf "East\nWest\nEast\nNorth\n" > sandbox/regions.txt
cat sandbox/regions.txt
sort sandbox/regions.txt | uniq -c
wc -l sandbox/regions.txt
history | tail
printf "one\n" > sandbox/output.txt
printf "two\n" >> sandbox/output.txt
cat sandbox/output.txt
```

Read the commands like English

- The first `>` creates/replaces `regions.txt`.
- `sort` groups equal lines together, then `uniq -c` counts adjacent repeats.
- `wc -l` reports 4 physical lines.
- `history | tail` shows recent shell commands.
- `output.txt` ends with both 'one' and 'two' because the second write used `>>`.

Expected check

check	expected
East count	2
West/North count	1 each
regions.txt lines	4
output.txt	one + two

Common beginner traps

- Using > when you meant append and overwriting evidence.
- Treating wc -l on CSV as a guaranteed row count.
- Pasting secrets into commands and later forgetting they may be in history.

Now you do a similar task

Use a pipe to count repeated status values in a small text file you create. Redirect the result to sandbox/status_counts.txt, then use history or Ctrl+R to find one earlier safe command and explain it.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What is the difference between |, > and >>?

Answer in your own words before continuing.

DE06-04 - Permissions, packages, sudo and command safety

What you will be able to do

Understand Linux permissions and use sudo only when the command actually needs elevated privilege.

Why this matters

Random sudo/chmod fixes can create root-owned project files and confusing failures later.

Picture it this way

sudo is borrowing the building administrator badge for one command. You should know why the ordinary employee badge is insufficient before using it.

Start from zero

- **ls -l** displays permission bits and file ownership.
- **id** shows your user/group identity.
- **sudo** runs a command with elevated privilege according to system policy. Routine Git/Python edits inside your home should not need it.
- **chmod** changes permission bits. chmod 777 is not a universal fix and gives broad permissions.
- Installing system packages may legitimately need sudo. Project-level package installs inside a Python virtual environment generally do not.
- If a file unexpectedly belongs to root, first ask which earlier sudo command created it instead of adding more sudo.

Teacher demonstration

```
cd ~/de-course/m06
id
ls -ld .
ls -l sandbox/notes/original.txt
# Create a safe executable demo file:
printf '#!/usr/bin/env bash\necho "hello"\n' > sandbox/hello.sh
ls -l sandbox/hello.sh
chmod u+x sandbox/hello.sh
ls -l sandbox/hello.sh
./sandbox/hello.sh
```

Read the commands like English

- No sudo is required because the sandbox belongs to your user.
- chmod u+x grants execute permission to the owner only.
- The permission string changes to include x for the user/owner.
- Running the script prints hello.

Expected check

check	expected
project owner	your Linux user, not root
chmod change	owner execute bit added
script output	hello

Common beginner traps

- `sudo git ...` or `sudo pip ...` for ordinary home-folder work.
- `chmod 777` because a forum comment says it fixes everything.
- Running package-manager commands without recording why/environment changed.

Now you do a similar task

Inspect ownership/permissions of `~/de-course/m06` and the seed project after you copy it later. Write three examples: a command that should not need `sudo`, a system-level command that might, and why `chmod 777` is a bad default.

How to prove it

- Save commands + observed output.
- Save `pwd/git status/diff/log` evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Why can using `sudo` for normal project work create problems even if the command succeeds?

Answer in your own words before continuing.

DE06-05 - Git state: init, status, add, commit, diff and log

What you will be able to do

See the difference between working-tree changes, staged changes and committed history.

Why this matters

Git becomes predictable once you stop treating add/commit as magic save buttons and instead inspect state.

Picture it this way

Working tree = papers on your desk. Staging area = papers placed into the next snapshot envelope. Commit = sealed labeled snapshot in history.

Start from zero

- **git init** creates repository metadata in .git.
- **git status** is your first state diagnostic. Read it before add/commit.
- **git diff** shows unstaged tracked changes. **git diff --staged** shows what is currently selected for the next commit.
- **git add FILE** stages content. It does not create history by itself.
- **git commit** creates a snapshot with author/message.
- **git log --oneline --decorate** shows compact history.
- Prefer focused add commands while learning; do not habitually use `git add .` before inspecting what it includes.

Teacher demonstration

```
# Copy the supplied seed into ~/de-course/m06/stage06-git-lab first.
cd ~/de-course/m06/stage06-git-lab
git init
git status
git add README.md .gitignore requirements.txt src/pipeline.py data/raw/orders.csv docs/architecture.m
d SAFE_TO_DELETE_LATER.txt .env.example
git diff --staged
git commit -m "Initialize Stage 06 engineering lab"
# Make one small README edit, then:
git status
git diff
git add README.md
git diff --staged
git commit -m "Document local rerun steps"
git log --oneline --decorate
```

Read the commands like English

- First status shows untracked project files.
- After add, `git diff --staged` shows exactly what will enter the commit.
- The first commit captures the baseline project.
- The second commit should contain only the README change.
- A clean final git status proves there is no forgotten working-tree change.

Expected check

check	expected
focused commits	2
final status	working tree clean

check	expected
log	two readable commit messages

Common beginner traps

- git add . without reading status.
- Thinking add saved a permanent version.
- Committing .env, .venv, logs or generated output by accident.

Now you do a similar task

Initialize a fresh seed copy. Make two meaningful commits: baseline and one documentation improvement. Save git status before staging, git diff --staged before each commit, and the final log.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

If a file is modified but not staged, which command shows the change? What if it is staged?

Answer in your own words before continuing.

DE06-06 - Branches, merges and real conflict resolution

What you will be able to do

Create branches, merge them and resolve a deliberate content conflict based on intended final meaning.

Why this matters

Conflicts are normal when two histories edit the same lines. The important skill is judging correct final content, not deleting markers mechanically.

Picture it this way

Two editors change the same sentence in separate copies. Git cannot decide which sentence is the intended final version, so it highlights both and asks a human.

Start from zero

- A **branch** is a movable name pointing to commits.
- **git switch -c branch-name** creates and switches to a branch.
- A merge combines histories. If edits do not conflict, Git can often merge automatically.
- A conflict file contains markers: <<<<<<, =====, >>>>>>.
- Resolution means edit the file to the intended final content, remove markers, test/read the result, stage it and commit the merge.
- Before/after **git status** tells you whether a merge is in progress and whether resolution is complete.

Teacher demonstration

```
cd ~/de-course/m06/stage06-git-lab
git switch -c docs-update
# Edit the Architecture sentence to:
# Source CSV -> validate -> transform -> output.
git add docs/architecture.md
git commit -m "Describe validation flow"
git switch main
# Edit that SAME sentence differently:
# Source CSV -> inspect -> summarize -> output.
git add docs/architecture.md
git commit -m "Clarify local summary flow"
git merge docs-update
# Git should report a conflict in docs/architecture.md.
git status
cat docs/architecture.md
# Edit it to the final meaning you choose, remove ALL conflict markers.
git add docs/architecture.md
git commit -m "Resolve architecture workflow conflict"
git status
git log --oneline --graph --decorate --all
```

Read the commands like English

- Both branches change the same line differently, creating a controlled conflict.
- cat exposes the conflict markers.
- You must choose a meaningful final sentence - not blindly choose ours/theirs.
- Staging the resolved file tells Git the conflict is resolved.
- Final clean status + readable file + graph are the proof.

Expected check

checkpoint	expected
conflict	docs/architecture.md
after resolution	no conflict markers
final status	clean
history	both branch histories visible/merged

Common beginner traps

- Deleting markers without deciding intended content.
- Running random reset commands to escape the conflict.
- Resolving the file but forgetting git add before committing.

Now you do a similar task

Repeat a fresh deliberate conflict in a new seed copy or new file. Before resolving, explain what HEAD/current branch and incoming branch each wanted. Save final file + clean status + graph.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Why does a merge conflict not mean the repository is corrupted?

Answer in your own words before continuing.

DE06-07 - GitHub remotes: push, fetch, pull and tracking

What you will be able to do

Connect local commits to GitHub and distinguish push, fetch and pull by direction and effect.

Why this matters

Remote confusion causes unnecessary merges and fear. Each command has a precise job.

Picture it this way

GitHub is another copy of the repository. push sends your commits outward. fetch downloads knowledge about remote commits without changing your current files. pull fetches and then integrates.

Start from zero

- **origin** is only a conventional remote nickname.
- **git remote -v** shows remote addresses.
- **git push** publishes local commits/branch refs to the remote when allowed.
- **git fetch** downloads remote refs/objects but does not automatically merge them into your current branch.
- **git pull** normally fetches and integrates according to your configured strategy.
- Upstream/tracking lets Git know which remote branch your local branch corresponds to.
- For this module, use a separate practice repository - never create conflict/revert exercises inside the deployed DE Mentor course repository.
- If WSL GitHub authentication becomes a long blocker, perform this remote lab from Git Bash/Windows where your browser credential flow already works. The fetch/pull/push concepts are the same; keep Linux filesystem lessons in WSL.

Teacher demonstration

```
# Create an EMPTY GitHub practice repository in the browser, e.g. m06-git-practice.
# Do not initialize it with README if your local repository already has history.
git remote add origin https://github.com/YOUR-ACCOUNT/m06-git-practice.git
git remote -v
git push -u origin main
# Make a small remote-only README edit in GitHub browser and commit it there.
# Back locally:
git fetch origin
git log --oneline --decorate --all
# Only after inspecting:
git pull
git status
```

Read the commands like English

- remote add stores the remote URL under nickname origin.
- push -u publishes main and sets upstream tracking.
- The browser edit creates a commit that only the remote knows initially.
- fetch downloads that remote commit without immediately rewriting working files.
- pull integrates it after you decide integration is appropriate.

Expected check

action	effect
push	local commits published
fetch	remote knowledge updated; current working tree not automatically changed
pull	fetch + integrate
final status	clean and synchronized

Common beginner traps

- Using your real course deployment repo for destructive practice.
- Assuming fetch changes the working tree.
- Pulling with unresolved local/conflict state without first understanding status.

Now you do a similar task

Use a throwaway/practice GitHub repo. Push local main, create one remote-only documentation commit, fetch it, inspect all history, then integrate. Write three arrows showing push/fetch/pull direction.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What changes locally after fetch that does NOT necessarily change after fetch?

Answer in your own words before continuing.

DE06-08 - Pull requests and review workflow

What you will be able to do

Use a feature branch and pull request as a reviewable boundary instead of editing shared main blindly.

Why this matters

Professional changes should be inspectable and discussable before they become shared history.

Picture it this way

A pull request is a proposed patch with a review table, not merely a green merge button.

Start from zero

- A PR compares a source branch against a target branch such as main.
- Keep changes small enough that someone can understand intent in Files changed.
- A useful PR description says what changed, why, how it was tested and any risk.
- Read the diff yourself before requesting/approving merge.
- After remote merge, your local main may be behind; switch to main and pull/fetch-integrate before new work.
- Delete stale feature branches only after you know their work is safely merged/published.

Teacher demonstration

```
git switch main
git pull
git switch -c readme-improvement
# Edit README.md to add a reproducible run command.
git status
git diff
git add README.md
git commit -m "Document reproducible run command"
git push -u origin readme-improvement
# In GitHub browser:
# 1. Open Pull Request
# 2. Read Files changed
# 3. Add description: why + validation
# 4. Merge the PR
#
# Back locally:
git switch main
git pull
git log --oneline --decorate -5
```

Read the commands like English

- Feature branch isolates the proposed change.
- Local diff is reviewed before commit/push.
- GitHub PR shows the branch diff for review.
- After merge, local main must synchronize with remote main.

Expected check

checkpoint	expected
PR diff	only intended README change
PR description	intent + validation
after merge	local main contains merged commit

checkpoint	expected
status	clean

Common beginner traps

- Opening a PR containing unrelated generated files.
- Merging without reading Files changed.
- Continuing work on stale local main after remote merge.

Now you do a similar task

Open a small PR in your practice repo changing only docs/architecture.md or README.md. Review Files changed, merge, synchronize local main, and save evidence of final log/status.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What problem does a PR solve that 'I can push directly to main' does not?

Answer in your own words before continuing.

DE06-09 - Python virtual environments inside a repository

What you will be able to do

Create a project-local Python virtual environment and prove it can be deleted/rebuilt instead of committed.

Why this matters

Repositories should describe dependencies, not store a giant machine-specific environment folder.

Picture it this way

.venv is a disposable workshop made from the repository's instructions. Commit the blueprint (requirements), not the entire workshop.

Start from zero

- **python3 -m venv .venv** creates an isolated Python environment folder inside the project.
- Activate in Bash with **source .venv/bin/activate**.
- Install project dependencies while the environment is active; record them in requirements/pyproject according to the project.
- .venv/ belongs in .gitignore and should not appear in **git ls-files**.
- If venv creation reports that ensurepip/venv is unavailable, only then install the distro's python3-venv package as an environment repair.
- A reproducibility check is delete/recreate .venv from tracked instructions and run the project again.

Teacher demonstration

```
cd ~/de-course/m06/stage06-git-lab
python3 --version
python3 -m venv .venv
source .venv/bin/activate
which python
python --version
python src/pipeline.py
deactivate
git status
git check-ignore -v .venv/ || true
git ls-files | grep -E '^(^|/)\.venv/' || echo ".venv is not tracked"
```

Read the commands like English

- which python should point inside the project .venv while active.
- The seed has no third-party requirements, so you do not need random installs.
- git check-ignore shows which ignore rule covers .venv.
- git ls-files must not list .venv contents.

Expected check

check	expected
activated python	path contains .venv
pipeline	runs
git tracking	.venv absent
ignore	.gitignore explains why

Common beginner traps

- Committing .venv.
- Using sudo pip inside a project environment.
- Installing random packages because a tutorial used them.

Now you do a similar task

Create .venv, run the seed pipeline, prove .venv is ignored/not tracked, delete .venv only after deactivation, recreate it, and run again. Save the commands/results.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Why do we commit requirements.txt but not the .venv folder?

Answer in your own words before continuing.

DE06-10 - .gitignore, secrets and repository hygiene

What you will be able to do

Keep secrets, environments, logs and generated state out of Git and prove ignore rules are actually working.

Why this matters

A .gitignore line is useful only if sensitive/generated files are untracked and stay untracked.

Picture it this way

gitignore is a do-not-pack list. If a secret was already packed into a commit, adding it to the list later does not magically remove the old copy.

Start from zero

- .gitignore patterns affect untracked files; they do not erase already tracked history.
- The seed ignores .env, .venv, logs, runtime state and common key files.
- **git status --ignored** can show ignored files; **git check-ignore -v FILE** shows the matching rule.
- **git ls-files** shows tracked files. This is strong evidence a secret/runtime file is not tracked.
- .env.example may be tracked because it contains safe variable names/placeholders only.
- If a real secret is ever committed, treat it as exposed: rotate/revoke it first, then repair repository history/policy with appropriate guidance. Do not merely add .gitignore and assume security is restored.

Teacher demonstration

```
cd ~/de-course/m06/stage06-git-lab
printf 'DEMO_API_KEY=fake-local-only-value\n' > .env
mkdir -p logs
printf 'local run log\n' > logs/run.log
git status --ignored
git check-ignore -v .env
git check-ignore -v logs/run.log
git ls-files | grep -E '^(^|/)\.env$|^(^|/))logs/' || echo "local secret/log files are not tracked"
git status
```

Read the commands like English

- The values are fake course data only.
- git check-ignore should point to matching .gitignore patterns.
- git ls-files should not reveal .env/logs as tracked.
- A clean status may still coexist with ignored local files.

Expected check

check	expected
.env	ignored, not tracked
logs/run.log	ignored, not tracked
.env.example	tracked safe template is okay

Common beginner traps

- Testing with a real API key/password.
- Believing .gitignore removes a secret already committed.
- Ignoring so broadly that important source/config templates disappear.

Now you do a similar task

Create fake .env + logs/runtime files, prove ignore rules with check-ignore and ls-files, then explain what you would do first if a REAL token had accidentally been committed.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Why is adding .env to .gitignore NOT enough after a real secret was already committed?

Answer in your own words before continuing.

DE06-11 - restore, revert, reflog and safe history repair

What you will be able to do

Undo the right thing at the right layer: working tree, staging area or committed shared history.

Why this matters

Recovery commands differ in scope. A beginner who understands state can repair mistakes without destructive guessing.

Picture it this way

A working-tree mistake is pencil on your desk copy. Staging mistake is putting the wrong page into the next envelope. A committed mistake is a sealed historical envelope - on shared history, revert creates a new correcting envelope.

Start from zero

- **git diff** inspects unstaged changes; **git diff --staged** inspects staged changes.
- **git restore FILE** can discard an unstaged tracked-file change. Inspect diff first because the change is lost.
- **git restore --staged FILE** removes a file from staging but keeps the working-tree edit.
- **git revert COMMIT** creates a new commit that reverses an earlier commit; it is usually safer for already shared history.
- **git reflog** records where local refs/HEAD have moved and can help locate commits after mistakes.
- Do not make **git reset --hard** your first recovery reflex. It can discard local work; this module teaches inspection + restore/revert first.

Teacher demonstration

```
cd ~/de-course/m06/stage06-git-lab
# Working-tree experiment:
echo "temporary local line" >> SAFE_TO_DELETE_LATER.txt
git diff -- SAFE_TO_DELETE_LATER.txt
git restore SAFE_TO_DELETE_LATER.txt
# Staging experiment:
echo "staged experiment" >> README.md
git add README.md
git diff --staged -- README.md
git restore --staged README.md
git diff -- README.md
git restore README.md
# History evidence:
git log --oneline -5
git reflog -5
```

Read the commands like English

- First restore discards only the inspected unstaged tracked-file edit.
- **restore --staged** moves the change out of staging but leaves it in working tree.
- Second restore discards the remaining working-tree experiment.
- **reflog** is evidence/history of local ref movement, not a command that changes anything by itself.

Expected check

scenario	tool
discard inspected unstaged edit	git restore FILE

scenario	tool
unstage but keep edit	git restore --staged FILE
undo shared bad commit	git revert COMMIT
find recent local HEAD movements	git reflog

Common beginner traps

- Using reset --hard before inspecting what would be lost.
- Confusing unstage with discard.
- Rewriting published history when a normal revert would preserve collaboration history.

Now you do a similar task

Create one harmless tracked-file edit and practise diff -> restore. Then stage a different edit and practise diff --staged -> restore --staged -> restore. Finally make a deliberate harmless commit and undo it with git revert; prove log/status.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What is the practical difference between git restore --staged FILE and git restore FILE?

Answer in your own words before continuing.

DE06-12 - Shell scripts, exit codes and environment-driven commands

What you will be able to do

Write a tiny shell script that fails clearly, returns useful exit codes and reads configuration from environment.

Why this matters

Data engineering automation depends on commands that signal success/failure to schedulers and other scripts.

Picture it this way

An exit code is a traffic light for the program that called your script: 0 means success; non-zero means something went wrong.

Start from zero

- A shell script starts with a shebang such as `#!/usr/bin/env bash`.
- `set -euo pipefail` is a common strict-mode pattern: stop on many command failures, unset variables, and pipe failures. Learn what it does rather than copying blindly.
- `$?` contains the previous command's exit status.
- `&&` runs the next command only if the first succeeded; `||` runs the next on failure.
- Environment variables let a script change behavior without hardcoding machine-specific values.
- Logs/messages should explain what failed without exposing secrets.

Teacher demonstration

```
# Create scripts/check_repo.sh:
#!/usr/bin/env bash
set -euo pipefail
PROJECT_ENV="${PROJECT_ENV:-development}"
echo "environment=$PROJECT_ENV"
test -f README.md || {
    echo "ERROR: README.md missing" >&2
    exit 2
}
python3 -m py_compile src/pipeline.py
python3 src/pipeline.py
echo "repository check passed"
# Then:
chmod u+x scripts/check_repo.sh
PROJECT_ENV=training ./scripts/check_repo.sh
echo $?
```

Read the commands like English

- `PROJECT_ENV` uses a safe default unless provided by the caller.
- `test -f` checks the required file.
- `>&2` sends the error message to stderr.
- `exit 2` gives a deliberate non-zero code if README is missing.
- With the seed intact, final `echo $?` should show 0.

Expected check

check	expected
intact seed exit	0

check	expected
missing README scenario	2
PROJECT_ENV	can be changed without editing script

Common beginner traps

- Using strict mode without understanding where failure is expected/handled.
- Returning success after a required check failed.
- Embedding secret values into a tracked script.

Now you do a similar task

Create `check_repo.sh`, run it successfully, then on a disposable copy rename `README.md` temporarily and prove the script exits non-zero. Restore `README` immediately and rerun successfully.

How to prove it

- Save commands + observed output.
- Save `pwd/git status/diff/log` evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

Why do schedulers/automation care about exit code 0 vs non-zero?

Answer in your own words before continuing.

DE06-13 - Repository handoff and clean-clone reproducibility

What you will be able to do

Prove another person can clone the repository into a clean folder and run it from tracked instructions.

Why this matters

'Works on my machine' is not a handoff. A repository should contain enough tracked instructions and dependency/config templates to reproduce the useful behavior.

Picture it this way

A reproducible repository is a flat-pack kit with all necessary instructions and labeled parts - not a finished workshop that depends on hidden tools under your desk.

Start from zero

- A **clean-clone test** catches hidden local dependencies and untracked instructions.
- README should explain prerequisites, setup, run command, input/output and safety notes.
- requirements.txt/pyproject describes dependencies; .env.example describes variable names without secrets.
- Generated logs/.venv/runtime state should be absent from Git.
- git status should be clean before handoff.
- Record enough evidence: commit hash, commands run, expected output and known limitation.

Teacher demonstration

```
# In a separate location, not inside the original repo:
cd ~/de-course/m06
git clone https://github.com/YOUR-ACCOUNT/m06-git-practice.git m06-clean-clone
cd m06-clean-clone
git status
git log -1 --oneline
cat README.md
python3 -m venv .venv
source .venv/bin/activate
python src/pipeline.py
deactivate
git status
git ls-files
```

Read the commands like English

- A separate clone prevents your original untracked files/config from helping accidentally.
- README becomes the only guide you should need.
- A new .venv proves the environment is rebuildable.
- Final status should remain clean because .venv is ignored.
- If the run needs a hidden file that README/.env.example never mentioned, the handoff is incomplete.

Expected check

checkpoint	expected
clean clone	new independent folder
run	pipeline prints description + input path

checkpoint	expected
final status	clean
hidden dependencies	none required beyond documented prerequisites

Common beginner traps

- Testing handoff in the same working folder where hidden files already exist.
- Forgetting to synchronize latest remote main before cloning.
- Writing README steps that you never actually tested from a clean state.

Now you do a similar task

Perform a clean-clone test of your practice repo. Follow only README instructions. Save the commit hash, exact run command, output, final status and one improvement you made after the clean-clone exposed a gap.

How to prove it

- Save commands + observed output.
- Save pwd/git status/diff/log evidence where relevant.
- Confirm you worked only in the intended sandbox/practice repo.
- Explain one failure mode before move-on.

STOP - check your understanding

What does a clean-clone test prove that a successful run in your original folder does not?

Answer in your own words before continuing.